

Algoritmos y Estructuras de Datos III

Trabajo Práctico N° 2	Unidad 2
Modalidad: Semi -Presencial	Estratégica Didáctica: Trabajo individual
Metodología de Desarrollo: Det. docente	Metodología de Corrección: Vía Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	Fecha Entrega: A confirmar por el Docente.

MARCO TEÓRICO (COMPLEJIDAD ALGORÍTMICA):

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

Parte I – Introducción a la Materia.

1. Qué entiende por Recursión

La recursión es una modalidad para abordar un problema dividiéndolo en versiones más pequeñas de ese mismo problema. En otras palabras, una función se invoca a sí misma para facilitar la tarea, paso a paso.

Para que no se quede atrapada en un bucle infinito, necesita tres regulaciones.

2. Que es una función Recursiva y que entiende por Recurrencia , dar ejemplos.

Una función recursiva es aquella que, dentro de su propio cuerpo, se llama a sí misma (directa o indirectamente) con un argumento modificado que la acerca al caso base. Cuando se invoca, el sistema apila el estado actual (variables locales y dirección de retorno) y comienza una nueva ejecución de la función con los nuevos parámetros.

La relación de recurrencia es la expresión matemática que describe este comportamiento. Formaliza cómo el resultado de un problema de tamaño n depende del resultado de un problema de tamaño $n-1$ (o menor).

Ejemplos:

Factorial: $n! = n \times (n-1)!$ con caso base $0! = 1$

Fibonacci: $F(n) = F(n-1) + F(n-2)$ con casos base $F(0)=0$, $F(1)=1$

Suma de naturales: $\text{suma}(n) = n + \text{suma}(n-1)$ con caso base $\text{suma}(0) = 0$

```
int factorial(int n) {  
    if (n <= 1) return 1;           // caso base  
    return n * factorial(n - 1);    // relación de recurrencia  
}
```

3. Explicar el uso del Stack del sistema operativo en Funciones Recursivas, dar un ejemplo.

Cada vez que se realiza una llamada a función, el sistema operativo reserva un bloque de memoria en la pila de ejecución "call stack", llamado registro de activación o stack frame. Este registro contiene:

- Los valores de los parámetros de la función.
- Las variables locales declaradas dentro de la función.
- La dirección de retorno (a dónde volver cuando la función termine).
- El valor de retorno (si aplica).

En una función recursiva, cada llamada genera un nuevo frame apilado sobre el anterior. Esto ocurre durante la fase de bobinado "winding". Cuando se alcanza el caso base, la pila comienza a desapilar en la fase de desenrollado "unwinding", resolviendo cada llamada en orden inverso.

Ejemplo con factorial(3):

```
factorial(3)
└ 3 * factorial(2)      ← frame 1 apilado
    └ 2 * factorial(1)  ← frame 2 apilado
        └ return 1      ← caso base (desenrolla)
            └ 2 * 1 = 2  ← frame 2 desapilado
└ 3 * 2 = 6             ← frame 1 desapilado
```

4. 4. Explicar las distintas variantes de Recursión.

Recursión directa: la función se llama a sí misma explícitamente dentro de su cuerpo.

Recursión indirecta (mutua): la función A llama a B, y B llama a A.

Recursión de cola (tail recursion): la llamada recursiva es la última operación de la función; no quedan operaciones pendientes al retornar. Los compiladores modernos pueden optimizarla eliminando el stack frame.

Recursión lineal: cada llamada genera exactamente una nueva llamada recursiva. El árbol de llamadas es una línea.

Recursión árbol (tree recursión): cada llamada genera dos o más llamadas recursivas. El árbol de llamadas se ramifica exponencialmente.

5. Describir las diferencias entre Iteración y Recursión.

La **Iteración** usa estructuras de control "for, while". El estado se mantiene en variables explícitas. No genera stack frames adicionales. Generalmente más eficiente en memoria. Mientras que, la **Recursión** es la función se llama a sí misma. El estado se mantiene en la pila. Cada llamada ocupa memoria en el stack. Código más compacto y legible para problemas como árboles, grafos o Hanói. Riesgo de stack overflow.

Ambas son equivalentes en poder expresivo, es decir, cualquier algoritmo recursivo puede escribirse de forma iterativa y viceversa. La elección depende de la legibilidad, el rendimiento requerido y la naturaleza del problema. Para problemas con estructura recursiva inherente, la recursión suele ser más natural.

MARCO PRÁCTICO

1. Para el Ejercicio de la Unidad anterior: "Un Negocio necesita gestionar la atención a sus cliente, el mismo recibe los clientes en un Box de Atencia que registra los clientes y los carga para su atención según su llegada.

Nos piden desarrollar un sistema de "Turnos" que se muestren en una pantalla.

La aplicación deberá registrar los clientes, mostrarlos en orden de llegada, llamarlos según ese orden por pantalla con una opción "próximo cliente – Box nro"

Nos Solicitan imprimir el tiempo de "Espera" de cada persona en la Lista, deberá Implementar esto de Forma recursiva.

2. Escribir dos funciones mutuamente recursivas par(n) e impar(n) que determinen la paridad del número natural dado, conociendo solo que:

A. 1 es impar.

B. Si un número es impar, su antecesor es par; y viceversa.

https://github.com/FernandezPegoRocio/-Algoritmos_y_Estructuras-de-Datos_III